

機械学習における計算軽量化の試み

静岡県立沼津東高等学校

理数科 2 年 岩崎辰海 大嶽利来 高橋寿々菜 中野秀志郎 吉末圭佑

担当教員 吉田直樹 渡邊朗仁

I 背景

近年 AI は目覚ましい発展を遂げている。そのなかでも、深層学習を用いた機械学習のモデルは日に日にその規模が大きくなっており、メモリ消費、エネルギー消費などの面で課題が生じている。そこで本研究ではこれらに用いられるニューラルネットワークの構造に着目した。ニューラルネットワークはあらゆる AI モデルに組み込まれており、非常に大きな計算コストを要し、多くの電力を消費する。我々はこのニューラルネットワークの構造の一部を削除し、軽量化することで、これらの問題を解消することを目指した。この手法は一般的にプルーニングと呼ばれ、大規模なネットワークにおいて広く用いられている。

従来のプルーニングの手法として、重みの絶対値の大きさに基づき、ノードやフィルタを対象にした削減が行われる。絶対値での評価はそれぞれのノードやフィルタが後の計算にどれだけ影響をあたえるかという観点によるもので、これらの手法は非常に有効であることが示されているが、モデルの効率化のために他の評価方法があるのではないかと考え本研究を開始した。

II 前提知識

本研究では、画像分類を行い入力された画像が正しく認識され分類されるかを評価した。

・学習

画像などのデータを入力しモデルに傾向をつかませるプロセス。

・推論

学習させたモデルに未知のデータを入力し正しい回答ができるか確かめるプロセス。

・ニューラルネットワーク (NN)

学習、推論を行うことができる、機械学習モデル複数のノードからなるレイヤーの層状構造になっており数字として入力されたデータを加工し次の層へ伝達した加工することを繰り返すことでデータの特徴を抽出する。

・ノード

NN で前の層からデータを受け取り次の層へ受け渡すもの。(図 1)

・畳み込みニューラルネットワーク (CNN)

畳み込み処理により画像から特徴を抽出しやすくしたものを NN に入力する。

本研究では CNN の学習、推論を行い、その結果を研究におけるデータとして使用する。

使用するデータセット (学習、推論に用いる画像)

MNIST: 0~9 の数字が書かれた白黒のデータセット

Cifar-10: 様々な物体のカラーデータセット

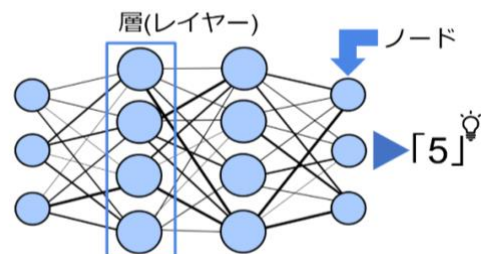


図1 ニューラルネットワーク

CNN の全結合層では各層で、各ノードに前のノードから数値が入力される。前層のノードが保持する値にそれぞれの“重み”をかけ、次のすべてのノードに対し出力を行う。次のノードに出力された値は各ノードで足し合わせられ、活性化関数により処理される。この計算を繰り返すことで、推論を行っている。

M 層目の入力値から次の、 $M + 1$ 層目のノードの保持するベクトル X_{M+1} を求める過程を以下に示す。

$$Y = X_M W_M + \beta_M \quad (1)$$

$X_M = (x_1 \ x_2 \ \dots \ x_m) \in \mathbb{R}^m$ は層 M に入力されたベクトル

(m, n はそれぞれ層 $M, M + 1$ におけるノード数)

x_k は層 M のノード k の入力を表すスカラー

$W_M \in \mathbb{R}^{m \times n}$ は層 M の重み行列

$\beta_M \in \mathbb{R}^n$ は層 M のバイアスのベクトル

$Y = (y_1 \ y_2 \ \dots \ y_n) \in \mathbb{R}^n$

したがって、(1) 式を成分表示したものにすると以下の通りとなる。

$$(x_1 \ x_2 \ \dots \ x_m) \begin{pmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{pmatrix} + (\beta_1 \ \beta_2 \ \dots \ \beta_n) = (y_1 \ y_2 \ \dots \ y_n) \quad (2)$$

さらに、活性化関数で Y に加工を施す。画像分類モデルでは下記 (3) 式の ReLU 関数を使用するのが一般的であるため、本研究でもそれに倣って ReLU 関数を活性化関数として用いる。

$$y'_l = \begin{cases} y_l & (y_l \geq 0) \\ 0 & (y_l < 0) \end{cases} \quad (3)$$

$$X_{M+1} = (y'_1 \ y'_2 \ \dots \ y'_n) \quad (4)$$

この操作を繰り返すことで最終的な出力値が得られる。

III 研究目的

本研究では、新たなプルーニング手法「Toba」アルゴリズムを提案する。一般的な CNN の構造を持つネットワークに対し、Toba を適用することで、性能を維持しつつ、ネットワークを軽量化することを目指す。

IV 実験に用いる手法

本実験では、従来のプルーニング手法に代わる手法として以下に示すアルゴリズムをに実装した。(以降このプルーニング手法を“Toba”と呼称する。)

Toba アルゴリズムでは、各ノードから出力される数値の類似度に着目する。出力値が類似していれば、そのノードの推論結果に対して果たす働きも類似しており、片方のノードを削除し

でもネットワークの性能を維持できると考えた。

類似度を測る方法として、各層でノードから出力された値の組から相関係数を用い、その値が大きい組のうち片方のノードを削除する。また、削除したノードの削除前に元々持っていた機能を損なわないよう、削除しない方のノードの重みに、補完する。

▼Toba アルゴリズムのフローチャート

-
1. 対象の層の出力する値を計算
 2. 以下を各層の持つノードのすべての組み合わせで繰り返す
 3. |--比較する 2 つのノードの出力値を抽出
 4. |--相関係数を導出
 5. 相関係数を絶対値の大きい順にソート
 6. 指定された個数のノードを上から順に削除
 7. 削除しなかったほうのノードの重みを補完
-

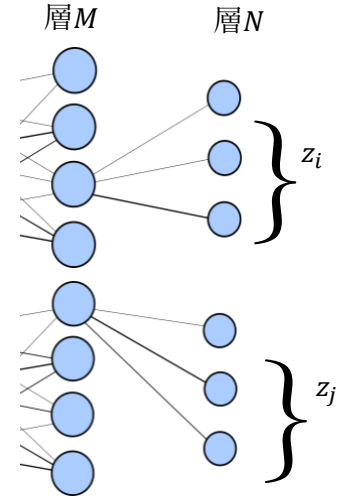


図 2

層 M において上記の Toba を行うときの処理を以下に記述する。
フローチャートの行番号と以下に振った番号は対応している。

1. 層 M (全結合) の出力ベクトル z_k は以下の様に表される。

$$z_k = x_k W_{Mk} \in \mathbb{R}^n \quad (5)$$

* (1) で示した Y は (6) 式のようにこの値を統合したものであるため、 $z_1 \dots z_m$ とは異なる

$$Y = \sum_{k=1}^m z_k \quad (6)$$

- 2,3. 削除するノードを i 、これと比較するノードを j とし、 $i \neq j \quad i, j \in \{1, 2, \dots, m\}$ となる

ような i, j の組を指定。組の個数は ${}_m C_2 = \frac{m \cdot (m-1)}{2}$

4. z_i と z_j の間の相関係数 r_{ij} を求める

$$r_{ij} = \frac{Cov(z_i, z_j)}{\sigma(z_i) \cdot \sigma(z_j)} \quad (7)$$

$Cov(z_i, z_j)$ は共分散, $\sigma(z_i), \sigma(z_j)$ は標準偏差を表す。

5. r_{ij} の絶対値が大きい順番にそれらに対応した (i, j) の組を取り出す。ただし、削除されるノードは、すでに削除または補完対象として確定したノードに含まれない
(二重での削除、別のノードの補完に使用されるノードの削除の禁止)

6. W_M から i 行目を削除, W_{M-1} の i 列目を削除, β_{M-1} から β_{M-1i} を削除

7. 重みの補完には相関係数の示す線形性を活かし直線的に近似する。ノード i を削除しノード j に補完するとき、最小二乗法を用い、 $z_i \approx az_j + b$ となるような a, b の値を求め、 $z_i + z_j \approx (a + 1)z_j + b$ とし、重みを補完する。(8) 式では z_i, z_j の平均をそれぞれ μ_i, μ_j としている。

$$a = \frac{\text{Cov}(z_i, z_j)}{\sigma(z_j)^2}, b = a\mu_j - \mu_i \quad (8)$$

$$W_M \text{において、} W_{Mj} \leftarrow (a + 1)W_{Mj} \\ \beta_M \leftarrow \beta_M + b$$

一連の処理は、複数の入力画像を用いて実行する。

以上のアルゴリズムを実装し、実験に用いた。実装には Python を用い、データ処理、モデル構築に NumPy を使用した。

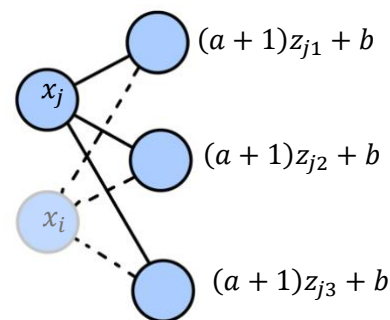


図 3

V 効果の検証

(1) 目的

IVで実装した Toba を実際に推論に用いたとき、その有効性を検証する。

(2) 設定

学習済みの CNN モデル*を用いて、以下の 3 つのプルーニングを適用し MNIST, CIFER-10 の推論を以下の条件で行う。

1. ランダムでノードを削除したモデル（以降ランダムと呼称）
2. Toba を適用したモデル
3. Toba を適用した後ある程度追加で学習させたモデル（実用段階と同様の設定にするため）

プルーニング処理を行う前の学習済みモデルは 3 つとも同じものを用い、ノード削除率を変化させ正答率、計算量を調べる。Toba は、Dense 層の 3 つすべての層に対し適用する。（畳み込み層、最大プーリング層、平坦化層においては実行しない。）また、Toba を実行するすべての層において同じ割合で削除を行う。

学習→1~3 のプルーニングをそれぞれ行う→評価 を 1 サイクルとして、MNIST では 80 サイクル、CIFER-10 では 20 サイクル行い、平均をとった。

*実験に用いるネットワーク

①構造について

畳み込み（Convolution）層を Conv と略して表記している。

畳み込み層はいずれもフィルタサイズ 3, ストライド 1, パディング 1

最大プーリング層はいずれもフィルタサイズ 2, ストライド 2, パディング 0
※ただし、そのままではフィルタが収まりきらない場合にはパディングを行っている。
(MNIST の最大プーリング層 3 層目が該当)

Mnist:

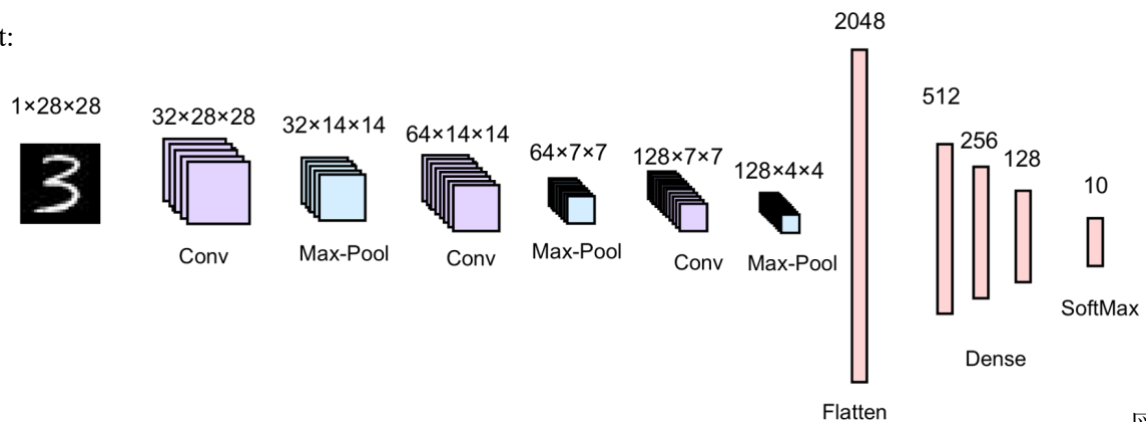


図 4

CIFAR-10:

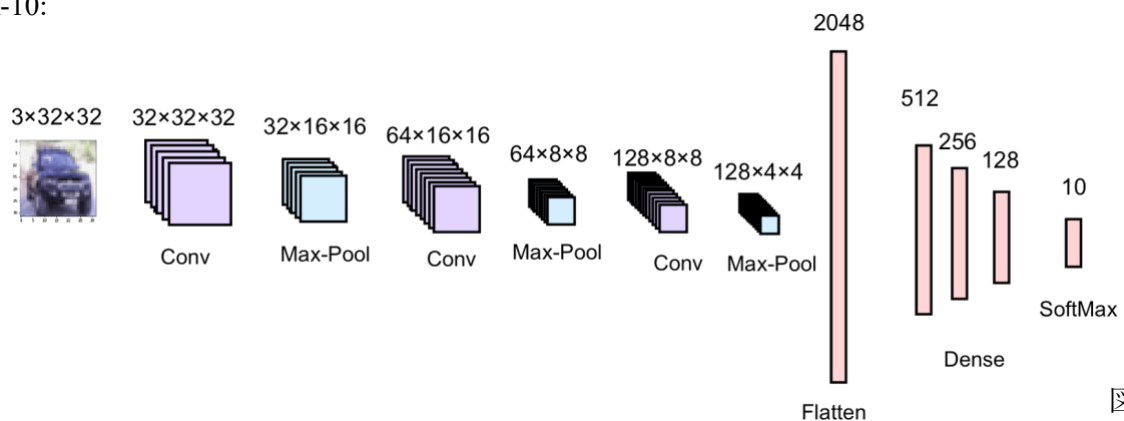


図 5

Mnist、CIFAR-10 のいずれのモデルにもドロップアウト、L2 正則化を適用し、各畳み込み層の後にバッチ正規化を行なっている。

- ・ドロップアウト

学習時にそれぞれのノードが指定された確率で無効化され、ノードの持つ値が 0 になる。推論時にはノードの無効化はされないが、その分次の層に渡す値が大きくなってしまうため無効化されないノードの割合(1 - (ドロップアウト率))を各ノードが保持する値に乗算する。

- ・L2 正則化

学習時にそれぞれのパラメーター (フィルタ、重み、バイアス等) の平方が小さくなるようにし、0 にある程度近づける。正則化係数によって平方の値を小さくしようとする作用の強さを調節する。

(本実験では、ドロップアウト率 Conv: 0.26、Dense: 0.33 正則化係数: 0.0005 で実装)

- ・バッチ正規化

層が持つ値の分布の平均と標準偏差を変更する。どのような平均と標準偏差にするかは学習によって決定される。

②その他の設定について

- epoch: 初めの学習については、MNIST は 10, cifar-10 は 40、追加学習については、MNIST は 4, cifar-10 は 20
- 最適化手法: Adam
- 学習率: 0.002
- ミニバッチのサイズ: 64
- 学習率スケジューラ: 1 epoch 毎に学習率を 0.92 倍

(3) 実験結果

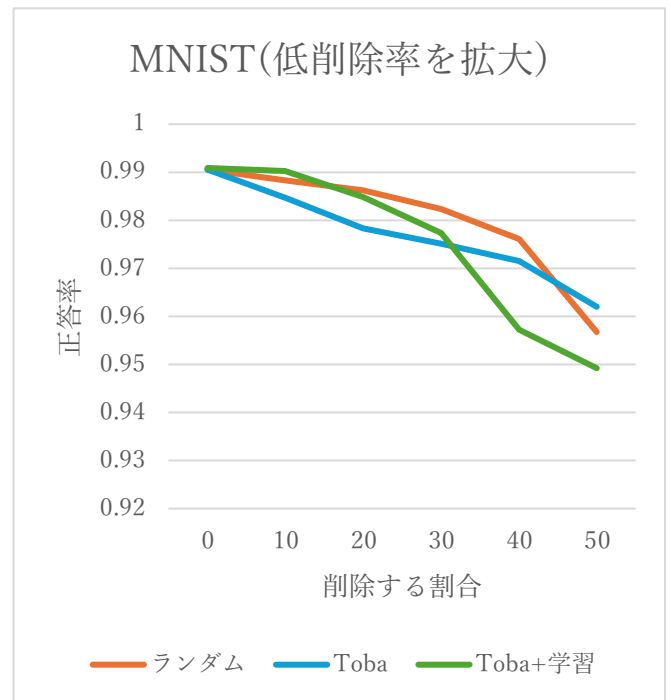
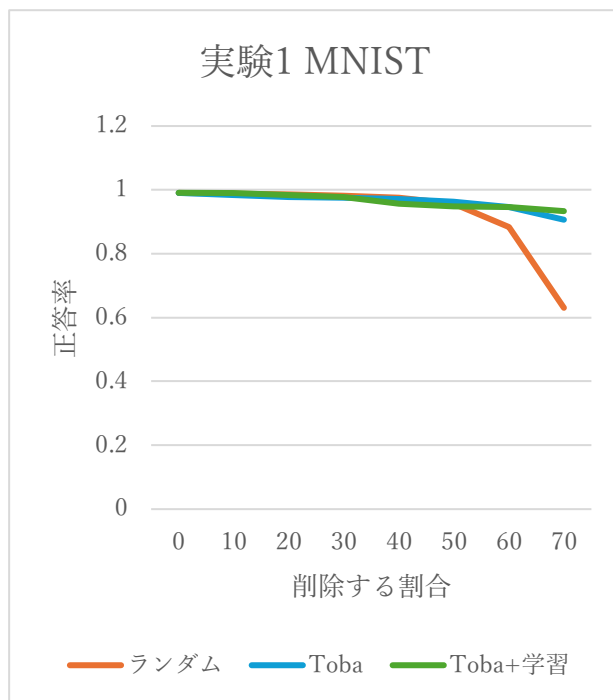
- ① 削除割合と計算量の関係 (計算量は、一枚の画像の推論における Dense 層～出力層までの値)

削除率	0	10	20	30	40	50	60	70	(%)
計算量	2.4	2.2	1.9	1.6	1.3	1.1	0.9	0.7	(MFLOPs)

※FLOPs とは、浮動小数点演算の回数の合計。1 MFLOPs= 10^6 FLOPs

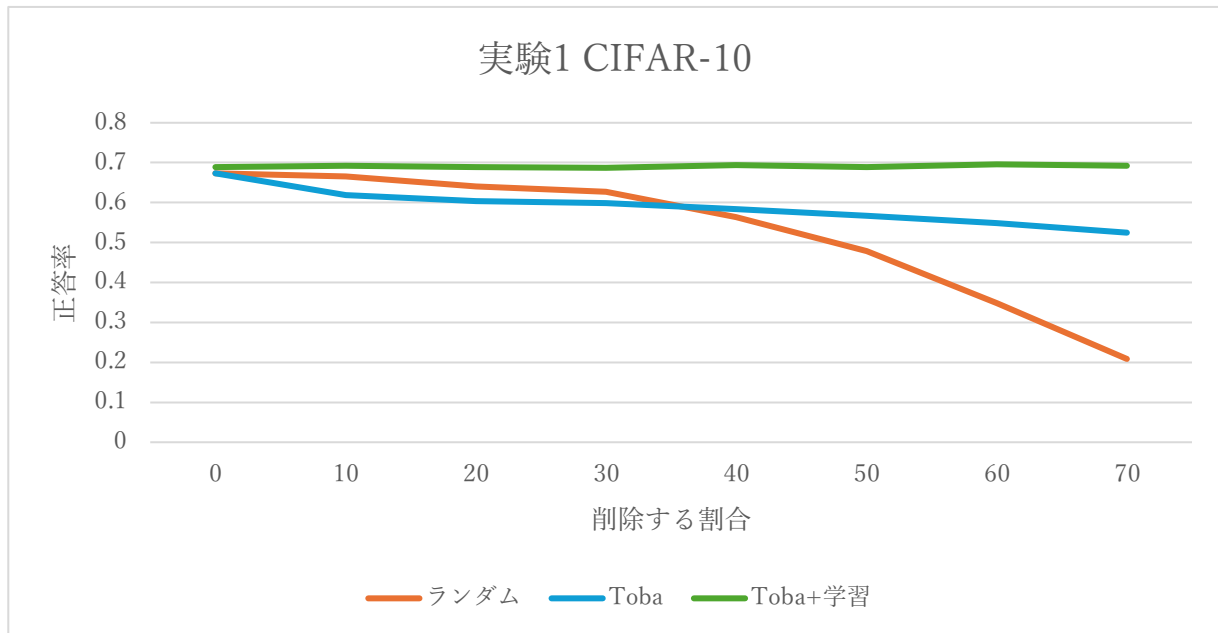
- 全結合層では、理論上計算量は削除率の 2 乗に比例する。
- 全結合層の計算コストは非常に高く、削除を行うことで計算量の問題に対処できることがわかった。

- ② MNIST における削除割合と正答率 (Top-1 accuracy)



- 削除率が 30%以下の場合、Toba 適用モデルは、ランダムと比べ若干低い結果となった。
- 削除率が 40%以上になると、Toba 適用モデルの方が、ランダムと比べ、高い正答率を維持した。
- Toba モデルと Toba 適用後ある程度学習させたモデルに有意な差は見られなかった

③ CIFAR-10 における削除割合と正答率 (Top-1 accuracy)



グラフ 3

- ・ランダム、Toba 適用モデルの精度の傾向は、Mnist と同じものが得られた。
- ・Toba を適用した後ある程度学習させたモデルでは、モデルの性能を完全に維持しており、正答率が上昇することも確認することができた。削除率 70%に着目すると、全結合層での計算量を元の 30%程度まで減らしつつ、正答率の低下を Mnist では 0.1、CIFER-10 では 0.2 程度に抑えることが確認でき、Toba が有効であることが示された。

(3) 考察

1. 低削除率における、Toba アルゴリズムの性能低下

低削除率（30%以下）においては、ランダムが非常に高い精度を維持している。また、Toba 適用モデルでは、削除を始めた段階で一気に正答率が下がっている。これは、学習時のドロップアウトの適用や、Toba アルゴリズムのノード削除の傾向が関わっていると考えられる。

2. 高削除率における Toba アルゴリズムの優位性

高削除率（40%以上）においては、ランダムの正答率が一気に低下した。40%以上の削除をランダムに行うとモデルの性能を維持できなくなることが確認された。Toba 適用モデルでは、低削除率よりは正答率は劣るが低下の傾向が緩やかになり、ある程度の性能が維持できていることが確認された。

3. 再学習の効果

再学習によって、特に CIFAR-10 では性能の維持、向上が確認された。

一般に、ネットワークの構造を複雑にするとより複雑な表現が可能になるが、一方で学習に用いたデータに過度に対応してしまい、テスト用の未知のデータに適応できなくなってしまう過学習（Over Fitting）という現象のリスクが高まるとされている。

Toba はノードを削除する事でネットワーク構造の簡略化を行なっているため、過学習の影響を軽減し、それによって精度の維持ないし向上が確認できたと考えられる。

VI Toba の課題

IVによって Toba の有効性が確認された。一方で、削除率が低いとランダム削除に正答率で劣る傾向が見られた。本項では Toba の問題点について考察、検証、対策の検討を行う。

(1) 原因の考察

もし z_i, z_j の値が全て等しく、偏差（平均との差）が全て 0 となった場合、標準偏差及び共分散が 0 となり $0 \div 0$ の形が発生してしまう。この対策としてネットワーク上では相関係数の計算時に分母に非常に小さな正の値 (10^{-8}) を加えているため、最終的に算出される相関係数の値は 0 となる。

本来このようなことが発生する確率は極めて低いが、先述の活性化関数によって、負の値の出力値を全て 0 にしているため、 $x_i, x_j = 0$ となる確率が高くなる。また、(5) 式のから、 $x_i, x_j = 0$ の場合 z_i, z_j の値が全て 0 となり、それに伴って $\sigma(z_i), \sigma(z_j) = 0$ となる。先行研究より、このようなノードは推論への影響がないため削除しても問題がないと考えることができ、従来のプルーニングではこれらに注目していた。ランダムではこれらのノードを確率的に一定量削除していると考えられる一方で Toba では一切削除をできないことで、最終的に発生する誤差の総量が Toba の方が多くなってしまいう事が原因であると考察した。

(2) 手法の拡張

① 0 禁止ランダム

ランダム削除においてデータセット中のどの画像を入力したときでも、 $x_k = 0$ となるようなノード k の削除を基本的に禁止する。ただし、そうでないノードを全て削除しても指定した削除率に達さない場合、0 をもつノードも削除する。

② 0 優先 Toba

z_i の値が全て 0 のとき、 z_j の値に関わらず (8) 式で $a = 0, b = 0$ とすれば、他の場合と同様に処理を行う事が可能である。その場合は (8) 式において $Cov(z_i, z_j) = 0$ であることから a が自動的に 0 となり、 b についても $a = 0, \mu_i = 0$ であることから自動的に 0 となる。ただし、相関係数の計算と同様に 0 除算を回避するために分母に非常に小さな正の値を足している。

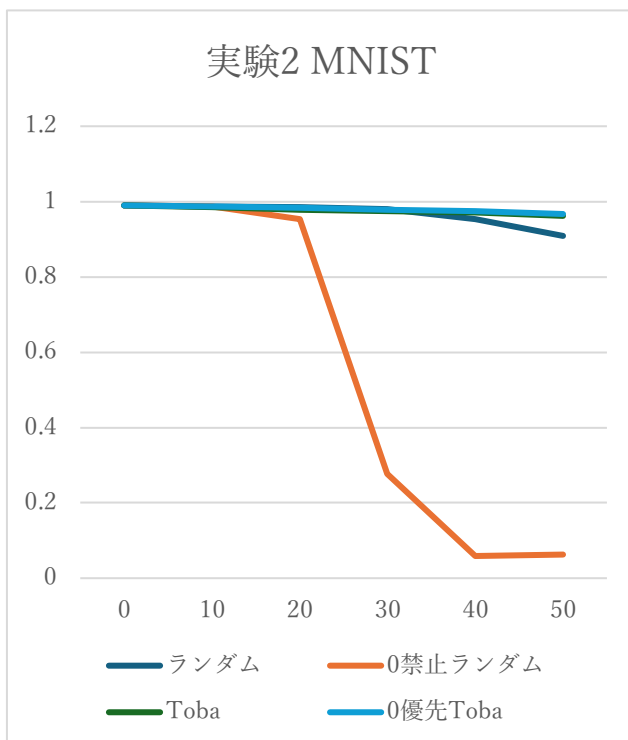
また、 z_i の値が全て 0 の場合にノード削除及び補完を行った場合は誤差が発生しないため、優先的に削除するようにする。これは従来のプルーニング手法を取り込んだものとも言える。

(3) 仮説の検証

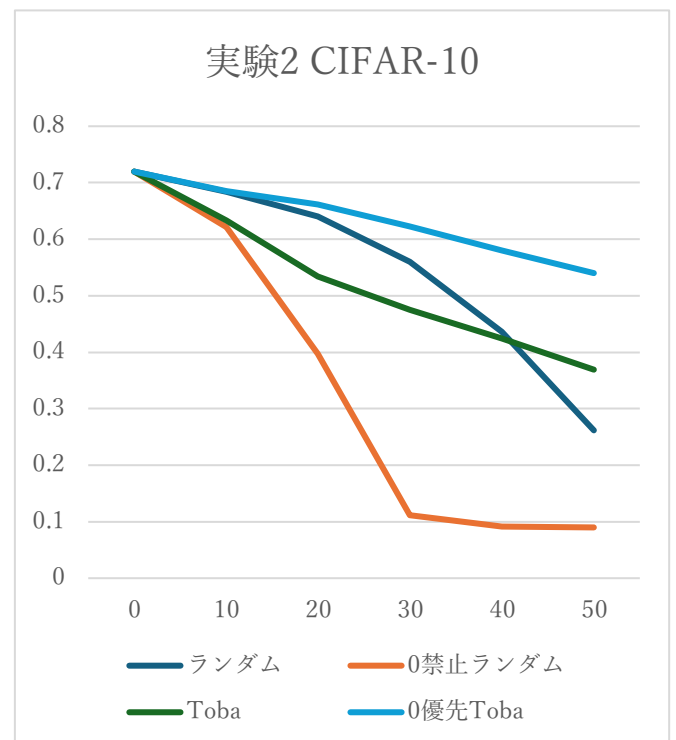
IVで行ったランダム、Toba に加え、(2) の 0 禁止ランダムと 0 優先 Toba の 4 つについて、IV と同じ内容の検証を行った。

MNIST での検証は 20 サイクル、CIFAR-10 での検証は 4 サイクル回した平均をとった。

(4) 検証結果



グラフ 4



グラフ 5

- ・ 0 禁止ランダムは精度が大幅に低下し、モデルの性能を全く維持することができないことが確認された。

- ・ 0 優先 Toba は他のどの手法よりも精度を維持することができた。低削除率においてもランダムより高い精度が出ており、高削除率でも Toba より優位な結果となった。

結果から (1) で考察した内容が、この問題の原因の一つになっていることが確認できた。また、0 優先 Toba にすることで、より高い効果を得られることも確認された。MNIST での結果は 0 優先 Toba とランダムにほとんど差異が見られなかったが、いずれもほとんど正答率の低下が見られず、MNIST がタスクとして簡単すぎるために発生したものだと考えた。

VII 結論と今後の展望

Toba によって、ある程度精度を維持しながら、ネットワークの構造を削減できることが確認できた。また、データセットによっては過学習の対策になることも示唆された。

通常の Toba では削除率が低いときにランダム削除に精度で劣っている問題があったが、従来のプルーニング手法を組み合わせることで精度をさらに向上させられる事がわかった。

今後はより複雑なデータセットやモデルでの検証を行っていきたい。

本研究で作成したコードや Excel ファイルは以下の Github リポジトリとして公開している。



<https://github.com/Tatchan123/Tankyu.git>

参考文献

斎藤 康毅 ゼロから作る Deep Learning

オライリージャパン 実践 Deep Learning

斎藤 正彦 線形代数入門

"Network Trimming: A Data-Driven Neuron Pruning Approach towards Efficient Deep Architectures"